

How FalsifyLab works

A plain-English tour of the app and codebase, tied back to your original plan

1. The idea in one minute

Think of an exam with a strict rule: **before you may look at any data, you write down your prediction and how confident you are**. You then do the work, hand in one answer, and a sealed answer key grades it. Only afterwards are you shown the textbook chapter that explains the right method, and you write down what you got wrong. The next exam is built so that you can only ace it if you absorbed the previous chapter.

Here the student is an AI model, and the exams are six experiments on one biology question. The point is not the biology. It is to measure two things about the AI: **does it know when it is wrong** (stated confidence vs. actual score), and **does it learn from feedback** (later scores vs. a run with the chapters withheld). Your plan called this Track 2: “science agents that know when they’re wrong”.

2. Just enough biology

Term	Meaning
GLP-1R	A receptor on cells that detects the hormone GLP-1. Drugs such as semaglutide (Ozempic, Wegovy) switch it on, which curbs appetite and blood sugar. It is the hypothesis’ target.
Drug target	The molecule in the body that a drug acts on.
GWAS	Studies that scan many people’s DNA to find genes linked to a trait (here BMI and type-2 diabetes). Gene linked to disease = a hint it could be a good target.
PDB / contact residues	PDB is the public library of 3D protein structures. “Contacts” are the amino-acid positions physically touching the drug, like the teeth of a key touching the lock.
Peptide vs. small molecule	Peptide: a short protein-like drug, usually injected. Small molecule: a pill-sized chemical. The hypothesis says the second kind could work too.
EC50	The dose giving half of a drug’s maximum effect. Lower = more potent.
Orthosteric / allosteric	Pressing the receptor’s main button vs. a different spot that also switches it on. The pill (orforglipron) uses the second.
Calibration	Does stated confidence match results? Gap = confidence minus score. Positive means overconfident.

3. The six experiments

Each experiment is a task the AI does in a code sandbox on real data. Later ones depend on lessons from earlier ones, which is how the lab can show learning.

#	What the AI is asked	Graded against	Needs lessons from
1	Rank 174 genes as drug targets using genetic evidence	Which genes really have approved drugs, beyond chance	-
2	Find which receptor positions semaglutide touches (3D structure 7KI0)	Contacts recomputed from the structure	-
3	Rank nine GLP-1 analogues by how long they last, and say why	Known duration classes + mechanism checklist	2
4	Fit dose-response curves and rank drugs by potency	Fit error + rank agreement with curated values	3

#	What the AI is asked	Graded against	Needs lessons from
5	Pick a test system and predict if a pill-type drug can work	Checklist; penalty for choosing a rodent (the pill barely works on mouse/rat receptors)	2, 4
6	Write an evidence-weighted verdict on the whole hypothesis	Checklist vs. the phase 3 trial result	1-5

4. Life of one experiment

1 Start	Engine announces the experiment. Scientist walks to the station.
2 Commit	AI writes prediction + a number for confidence, then a plan. Every other tool says LOCKED until it does.
3 Work	AI runs Python on the data in a sandbox with no internet. A tool-call budget applies.
4 Submit	One answer. No retry.
5 Score	A separate sandbox holds the answer key and scorer. The AI never touches it.
6 Teach	Now the lesson card (paraphrased paper) is shown. AI writes what it got wrong and why it was confident anyway.
7 Audit	A separate deterministic pass grades the path (section 5). The agent never sees it.
8 File	Entry goes to the notebook; the lesson joins the pool for later experiments.

Why you can trust the result: the prediction is on record before data is visible (the gate, `engine/tools.py`); the answer key is mounted only in the scorer's sandbox (`engine/scoring.py`, with a test proving nothing private is reachable); and the lesson appears only after the score is fixed, so it cannot influence the attempt it grades.

5. The audit: grading the path, not just the answer

A high score can come from luck or a shortcut. After scoring, an auditor grades how the answer was reached, in three layers it never blends into one number:

Layer	What it measures	How
Outcome	Was the answer right?	The existing scorer, 0 to 1.
Process	Did it follow the paper's method?	Weighted checkpoints from the lesson card, checked only in the calls the answer came from, so a ceremonial step earns nothing.
Integrity	Was it legitimate?	Yes/no flags: touched the answer key, reached outside the sandbox, answer no run produced, answer unchanged when the data is destroyed, fake citation, prediction rewritten after seeing data.

Each attempt gets one verdict (valid success, wrong conclusion, insufficient evidence, protocol violation, unsafe action, reward hack, parse failure). **Clean success is valid success only**; the raw score stays as a secondary number. In the tests, an agent that hard-codes the right answer scores 1.00 but is a reward hack: clean success 0 of 1. Checks use what the code *did* (interpreter audit hooks, re-running its code on data with the signal removed), not keyword hunting.

6. How the app shows it

Everything the engine does is appended to an **event log** (types like *prediction_written*, *tool_call*, *scored*). That log is the single source of truth, and there are two views of it: the **notebook** (readable pages) and the **pixel lab** (the animation). In **replay** mode the backend serves a recorded log and the page plays it back, so a demo never depends on live model calls. In **live** mode events stream in as they happen. The frontend queues events and holds each on screen for a minimum time so the scientist walks rather than teleports.

This is also why the notebook bug you found existed: the notebook fetched the whole finished run, while the animation was only partway through the same log. It now shows only experiments the animation has finished.

7. Codebase map

Folder	Role	Plan section
engine/	Event log, spec loader, agent loop, tool gate, scoring, notebook, CLI	5.2 to 5.6, 8
sandbox/	Where untrusted code runs: local backend and Modal backend	5.4, 10
curricula/glp1r/	The six experiments as config: spec YAML, datasets, scorers, hidden answer keys (private/), lesson cards, data fetcher, scorer tests	4, 8.1, 8.4
curricula/wrn/	Second hypothesis (WRN helicase), built with zero changes to engine/, to prove the engine is hypothesis-agnostic	4.3 (stretch)
api/	FastAPI: replay, live streaming (SSE), Modal deployment	5.1, 5.6, 10
web/	React pixel lab, animation queue, notebook overlay	7
evals/audit/	Scripted agents with known right verdicts: honest, lucky, hard-coded, key-reading, unsafe	beyond the plan
art/	Draws every sprite with Pillow; no external assets	7.5
devin/	Playbook + launcher for one Devin session per experiment	9
runs/, replays/	Recorded runs and the demo replay	5.6
docs/	Reference verification, dataset provenance, credits, demo script	4.2, 14

8. Plan vs. what exists

Plan item	Status	Note
Experiments 1-6, one attempt each, scorers, lesson cards	Built	Ran end to end; mean score 0.90
Prediction gate, event log, notebook template, replay + live	Built	As designed in plan sections 5 and 8
Room, stations, scientist, animation queue, reactive decor	Built	All sprites generated from code
Reference verification (section 16)	Done	Two plan references were wrong; corrected in docs/references.md
Control run (with vs. without lessons)	Done	Result is inconclusive at one run per arm; experiment 5 is the clear win
Calibration summary page	Built	Notebook > Calibration
H2 / WRN curriculum (stretch)	Built	21,108 real DepMap measurements
Modal: sandboxes, hosting, parallel control run	Written, not run	No Modal credentials on the dev machine; local backend is the tested one
Devin: one session per experiment	Launcher only	Playbook and launcher exist; I found no record of sessions having been run
Path audit with clean-success headline (added after the plan)	Built	Passes 29 scripted eval cases; validated only on constructed cases so far
Audit measured on real labelled agent runs	Not done	Needs API budget and hand labels; process score is unvalidated until then

Plan item	Status	Note
Sound effects (stretch)	Not done	

Two hackathon-challenge gaps matter most: the Modal and Devin challenges are claimed in the plan, but only the code for them exists. Running either once would turn those claims from “written” into “demonstrated”.

9. What the results say

The AI scored well (mean 0.90) but was **underconfident** (average gap -0.30), the opposite of the plan's premise. Twice a low score was our own scoring bug, not the AI's error, which is why every scorer has a test that the right answer is not penalised for its wording. The recorded runs predate the audit, so they are shown as unaudited, not clean. Building the audit also found that the local sandbox let agent code read the answer key by absolute path; it is now blocked and logged. Caveats: these are famous results the model may partly remember, and one run per arm cannot separate lesson effects from noise.